Half an Airing Counts as a View: A Coverage-Based Label, Habit-Based Candidates and a LightGBM Re-Ranker for TV Program Recommendation on sweet.tv

Student: Manuilov Pavel
Email: pimanuylov@edu.hse.ru
Team: Individual
Date: 2026-10-24
Assignment: Final exam — TV program recommendation on sweet.tv (Kaggle `sweettv-tv-program-recommender`)

Abstract — The task is to predict, for 1,260 test users, the five TV programmes they will watch in weeks 31-42 from a view log of weeks 11-30 and the broadcast schedule, in which only 10.8% of the airings of weeks 11-20 carry a programme id. I join views to airings by channel and interval overlap, count a programme as watched when one continuous view covers more than 50% of an airing, compare popularity, implicit ALS, content-based filtering, repeat models and a slot-habit model on one temporal hold-out, and re-rank their candidates with a LightGBM ensemble. The designated final submission scores 0.44493 private / 0.46071 public MAP@5 against 0.24426 / 0.25483 for popularity; on validation it reaches 0.4265 against 0.4080 for the best single model. Takeaway: the definition of the label mattered more than the model; my first, any-overlap label fitted three leaderboard points almost perfectly and was still wrong.

Index Terms — TV recommendation, implicit feedback, learning to rank, temporal validation, MAP@5

1. Introduction

Problem statement. sweet.tv published a view log (5,821,987 views of 4,914 users, 2020-03-09 to 2020-07-26) and three schedule files (weeks 11-20, 21-30, 31-42) [1]. A view has a user, a channel and a time interval; a programme exists only in the schedule. For each of the 1,260 users in `submission.csv` the system must output five `tv_show_id`s for weeks 31-42; the metric is MAP@5. Only labelled airings can become items: 10.76% of the schedule rows of weeks 11-20 have `tv_show_id` ≠ 0 (11.31% and 11.18% in the other two files, `docs/data_inspection.md`).

Motivation. Linear TV is watched by habit: the same news, series and slots, day after day. A recommender that only re-lists what a user already watches can still be useful, but it has to be measured on the right notion of "watched", and a model has to be judged on future weeks it has not seen.

Concrete example. Test user `2725163102947201526` watched 662.4 hours and touched 504 labelled programmes in weeks 11-20. For weeks 21-30 the final setup recommends *Сегодня*, *Факты*, *Утро с Украиной*, *Викна-Новости* and *ТСН*; two of them (*Сегодня*, *Утро с Украиной*) are in the user's true top-5, which gives AP@5 = 0.3333 (`reports/hand_check.md`).

Contributions.

- A reproducible pipeline from raw CSVs to a ranked list, with a checked join and a coverage-based label, and an honest record of how a wrong validation label was detected with one diagnostic submission (§5).
- A comparison of six single methods and two hybrids on the same hold-out, with a temporal check for every learned re-ranker, and a final hybrid that improves private MAP@5 from 0.40026 (best single-model submission) to 0.44493.

2. Related Work

Prior work consulted. The competition page [1] hosts the data and the leaderboard. A prize-winning write-up for this competition [2] counts a programme as watched only when more than 80% of it was seen without switching channels, uses a simple count-based baseline ("more than 50% of the show in a single sitting") and a second layer with LightGBM; I took the label idea and some feature ideas from it, no code. Implicit ALS [3] weights observed interactions by confidence instead of treating them as ratings; I used the `implicit` library [7]. The re-ranker is gradient boosting with LightGBM [4] and the LambdaRank objective [5], which optimizes a ranking metric directly. Cremonesi et al. [6] showed that a non-personalized popularity list is a strong top-N baseline, which is why every method here is compared with one.

Alternatives considered.

- *Association rules* (not implemented, discussed only conceptually): rules such as "watches A ⇒ watches B" capture co-viewing, but here the strongest signal is repeating the same programme, which a count already captures.
- *Personalized PageRank* on a co-view graph (not implemented, conceptual): diffusion spreads a user's history to neighbouring programmes, i.e. towards new items; on this data even ALS, which also generalizes from co-viewing, stayed below popularity (Table I).
- *LightFM*, used as a candidate source in [2]: not tried for lack of time; ALS and CBF suggested that similarity-based candidates add little here.

3. Method

Approach. Everything runs offline in Python (`src/`, one module per step, `python -m src.run_all` reproduces the tables, examples and figures). A temporal hold-out mirrors the real task: history weeks 11-20, target weeks 21-30. The final model is a second layer over candidates from simple habit-based models.

Pipeline.

1. **Join** (`build_interactions.py`): glue the three schedules (3,441,607 airings kept, end = start + `duration` in seconds); merge overlapping or touching views of the same user on the same channel (5,821,987 views → 5,441,773 intervals, viewing time −1.3%), so parallel devices do not count twice; split intervals at Monday 00:00 so every interaction belongs to one ISO week; join on `vsetv_id = channel_id` with interval overlap. Result: 3,026,937 labelled (user, airing, week) pairs; 52.89% of viewing time falls on labelled airings, 41.15% on `tv_show_id = 0`, 5.96% matches no airing.

2. **Label** (`coverage.py`): for every (user, airing) the share of the airing covered by the best *single* continuous view. Fig. 1 shows why a threshold is needed: most pairs are short glimpses, a second peak are complete airings. A programme's target count = airings with coverage > 50% (**cov50**); the user's top-5 by that count is the validation ground truth.
3. **Candidates**: top-50 per user from seven sources: repeat_cov (with and without its minimum-count filter), the > 30% coverage ranking, the earlier loyalty repeat, popularity, ALS and slot habits; 121 candidates per user contain 83.9% of the true top-5 (`docs/hybrid_v4_results.md`).
4. **Features** (53): coverage counts at 30/50/80% and their ranks, decayed days/airings/seconds, weeks since the last full watch, loyalty, ALS score and rank, CBF cosine, slot score and rank, share of the user's time on the programme's channel, time-of-day fit, airings in the target window and their change against the history, popularity, category and channel.
5. **Re-ranker**: LightGBM LambdaRank (graded by $\log_2$ of the count) and binary models, three seeds each, rank-averaged; trained on five sliding folds (11-17→18-20 … 11-27→28-30) with recency weights and cov80 labels (§3, decisions).
6. **Top-5**: the five best-scored candidates; if fewer than five exist, the shared popularity fallback fills the list.

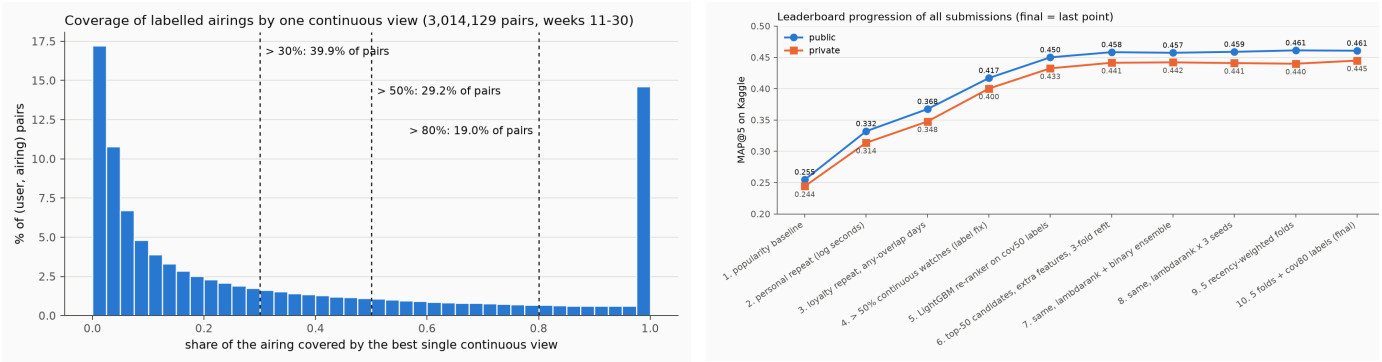


Fig. 1. Left: coverage of labelled airings by one continuous view (3,014,129 (user, airing) pairs, weeks 11-30; > 30 / 50 / 80%: 39.9 / 29.2 / 19.0% of pairs). Right: public and private score of all ten Kaggle submissions (Table II).

Tools & libraries. Python 3.11.9 on Windows 11 (Intel i7-11370H, 4 cores, 15.7 GB RAM); DuckDB 2.0.0 for joins [9]; pandas 3.1.0, NumPy 2.4.6, SciPy 1.17.1; `implicit` 0.7.3 [7]; LightGBM 4.7.0 [8]; scikit-learn 1.9.1 (TF-IDF); matplotlib 3.11.2. Pinned versions: `requirements.txt`.

Key design decisions.

- *A coverage label, chosen from the leaderboard, not from validation.* Validation can only rank models against a label I define; which label matches the hidden one can only be tested with submissions that disagree about it (§5). cov50 fits the four discriminating leaderboard points best of the candidates I tried; the final ranker is trained on cov80 labels, which the write-up names [2].
- *A strict reading of the test-schedule rule.* The weeks 31-42 schedule is used only for which programme airs on which channel and when (candidate pool, airing counts, slots). Content metadata comes only from schedules up to week 30, so test-only programmes have no CBF vector. No view after week 30 exists or is used.
- *Recency instead of more tuning.* The final ranker is trained on five sliding folds, weighted $0.5^{(28 - \text{first target week})/4}$ (0.18 … 1.0), and refit on all users with the validated number of trees.

Configuration. Popularity: watch seconds, last 10 history weeks. ALS: BM25-weighted decayed days, 8 factors, regularization 0.01, $\alpha = 0.3$, 15 iterations. CBF: TF-IDF over category, genres, decade, director and up to 5 actors per row (names in ≥ 2 programmes), people weight 4, user profile = weighted mean, cosine. repeat_cov: airings with > 30% coverage, half-life 4 weeks, loyalty scaling with $\beta = 10$, at least 3 such watches. Slot habits: channel × weekday/weekend × hour, half-life 8. LightGBM: learning rate 0.05, LambdaRank 15 leaves / 20 min. child samples, binary 63 / 100, subsample and column sample 0.8, early stopping (100 rounds) on 10% of training users. Seed 42 everywhere (ALS, LightGBM with `deterministic=True`, holdouts, samples); ensembles add 43 and 44. Runtime: `src.run_all` 1,397 s end to end, of which the final ensemble 1,263 s; every single method ≤ 2.3 s (`docs/runtimes.md`, Table I).

4. Experiments

Setup. Validation: history weeks 11-20, target weeks 21-30, the 1,260 test users (their weeks 21-30 views are training data, so they may be used). Candidate pool: the 6,376 labelled programmes airing in weeks 21-30, identical for every method; repeats of already-watched programmes are allowed. MAP@5 is the Kaggle formula (`src/metrics.py`); users with an empty ground truth count as 0. Every method outputs exactly five distinct ids: own ranking first, then the shared popularity fallback. Cold start: 10 test users have no labelled interaction in weeks 11-20 and 2 in weeks 11-30 (`docs/join_summary.md`); they get the fallback in the single methods. The hybrids never need it, because popularity is one of their candidate sources. A re-ranker's *temporal check* is the same model trained only on folds whose labels end before week 21. Hardware: as in §3.

TABLE I. ALL METHODS ON THE SAME HOLD-OUT (MAP@5, 1,260 TEST USERS, TARGET WEEKS 21-30; SOURCE `docs/results_table.csv`)

Method	Configuration	cov50	cov80	n_days	Temporal cov50	Temporal cov80	Runtime, s
popularity	watch seconds, last 10 history weeks	0.2752	0.2600	0.3167	–	–	0.0
ALS	implicit ALS, BM25, 8 factors, alpha 0.3	0.2528	0.2380	0.2927	–	–	1.2
CBF	TF-IDF metadata, people weight 4	0.1154	0.1068	0.1276	–	–	2.3
repeat_cont50	count of > 50% continuous watches (write-up Algorithm 1)	0.3841	0.3583	0.3690	–	–	0.2

repeat_cov	> 30% watches, half-life 4, loyalty beta 10, min 3 watches	0.4080	0.3748	0.4020	–	–	0.6
slot habits	channel x weekday/weekend x hour affinity	0.4015	0.3710	0.4041	–	–	2.1
hybrid_v2 setup	LightGBM lambdarank, 1 fold, top-20 candidates, cov50 labels	0.4263	0.3928	0.4228	0.4172	0.3872	59.5
final (hybrid_v4_cov80)	**LightGBM lambdarank + binary x 3 seeds, 5 recency-weighted folds, cov80 labels**	**0.4265**	**0.3977**	**0.4154**	**0.4229**	**0.3940**	**1263.0**

Comparison vs baseline. The final setup beats popularity by 0.151 on cov50 and the best single model (repeat_cov) by 0.018. Both required methods are below popularity: ALS by 0.022 and CBF by 0.160. Slot habits, which ignore programme identity and use only channel and hour, are almost as good as repeat_cov. On the leaderboard (Table II) the big steps (popularity → repeat → coverage-based repeat → hybrid) move with validation; the last six hybrids differ on validation by less than its run-to-run noise (§5), so validation cannot order them.

TABLE II. THE TEN KAGGLE SUBMISSIONS IN ORDER (VALIDATION = cov50, MAIN / TEMPORAL CHECK; SOURCE `submissions/LOG.md`)

#	Kaggle name → repo file	What changed	Val. main	Val. temporal	Public	Private
1	submission (2).csv → popularity_v1.csv	popularity baseline (watch seconds)	0.2752	–	0.25483	0.24426
2	submission.csv → repeat_v1.csv	personal repeat, log seconds	0.3335	–	0.33212	0.31372
3	submission (3).csv → repeat_v2.csv	loyalty repeat on any-overlap days	0.3576	–	0.36752	0.34770
4	submission (4).csv → repeat_cont50.csv	> 50% continuous watches (label test)	0.3841	–	0.41706	0.40026
5	submission (5).csv → hybrid_v2.csv	LightGBM re-ranker on cov50 labels	0.4271	0.4161	0.45010	0.43259
6	submission (7).csv → hybrid_v3_lr.csv	top-50 candidates, more features, 3-fold refit	0.4275	0.4203	0.45840	0.44146
7	submission (6).csv → hybrid_v3_ens.csv	same, LambdaRank + binary ensemble	0.4272	0.4228	0.45741	0.44213
8	submission (8).csv → hybrid_v3_lr_seeds.csv	same, LambdaRank × 3 seeds	0.4271	0.4204	0.45896	0.44098
9	submission (9).csv → hybrid_v4_recent.csv	5 recency-weighted folds	0.4270	0.4243	0.46126	0.43995
10	**submission (10).csv → hybrid_v4_cov80.csv**	**5 folds + cov80 labels (final)**	**0.4270**	**0.4210**	**0.46071**	**0.44493**

Final submission. Kaggle disables submission selection for late submissions, so I designate it here: **submission (10).csv = hybrid_v4_cov80.csv = submissions/final.csv, 0.46071 public / 0.44493 private**. It is the best private score of the ten; its public score is second to submission (9) (0.46126). The screenshot of the Submissions page is Fig. 2. Repository: github.com/ManuylovPI/rec_sys_mid_ex, code and results at commit `00d28ca9f4`.

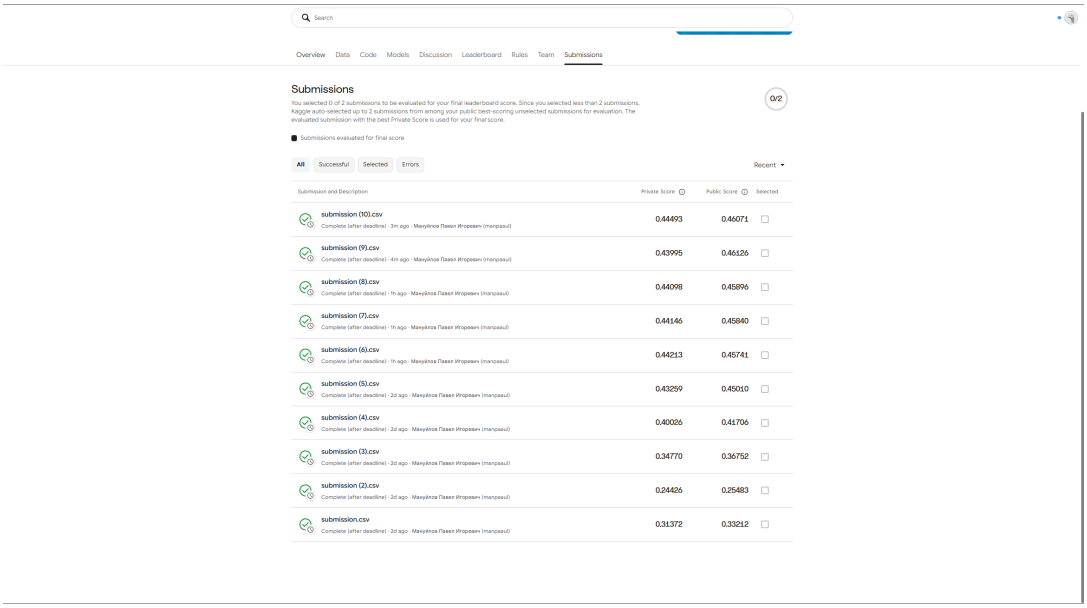


Fig. 2. Kaggle Submissions page with all ten scored submissions (reports/figures/kaggle_submissions.png).

Verification.

- *Join:* a brute-force overlap join on 284 sampled users gives the same 610,862 pairs and the same overlap seconds as the bucketed join, 0 mismatched pairs (`docs/join_verification.md`).
- *Metric:* `tests/test_metrics.py`, 6 hand-computed cases (e.g. hits at ranks 1 and 3 of 3 relevant → 5/9), all pass.
- *Predictions written before a submission:* for repeat_cont50 I recorded 0.298 (any-overlap label), 0.380 (cov50) and 0.387 (cov80) before submitting; it scored 0.417 public (`docs/repeat_cont_results.md`, §5).
- *Format:* every file is checked for header, 1,260 rows in template order and five distinct non-zero ids from the test pool (`src/submission.py`).
- *Reproducibility:* re-running every method from scratch reproduces all six single methods exactly; the two LightGBM hybrids vary by about ±0.001 (§5).

- *Hand check:* **[PAVEL: AP@5 for user 2725163102947201526 computed by hand in reports/hand_check.md = _____; code value 0.333333; match: _____]**

4.1 Two users and the strategy question

I chose the users by a fixed rule (`src/examples.py`): a heavy viewer closest to the 90th percentile of history viewing time, and a light viewer with a thin history (1-20 labelled programmes touched in weeks 11-20 against a median of about 200; the one with the most target programmes). Both have five target programmes.

TABLE III. TOP-5 PER METHOD FOR THE SAME TWO USERS, WEEKS 21-30 (ID TITLE [CATEGORY]; **BOLD** = IN THE USER'S cov50 TOP-5, * = WATCHED IN WEEKS 11-20, † = FALLBACK SLOT; SOURCE `docs/examples.md`)

List	1	2	3	4	5
Heavy viewer: user 2725163102947201526 — History weeks 11-20: 662.4 viewing hours, 504 distinct labelled shows watched. Target weeks 21-30: 5 shows in the cov50 top-5, 5 in the cov80 top-5.					
actual cov50 top-5	**12002378 М/с 'Удивительный мир Гамбол... [Детям]**	**1000589 Т/с 'Отчаянные домохозяйки',... [Сериалы]***	**2400508 Сегодня. [Инфо]***	**5900256 Утро с Украиной. [Инфо]***	**12002528 М/с 'Время приключений'. [Детям]**
popularity	2400480 ТСН. [Инфо]*	240081 Факты. [Инфо]*	**2400508 Сегодня. [Инфо]***	20088 'Завтрак с 1+1'. Информацион... [Развлечения]*	6500479 'Мир наизнанку'. [Познавательное]*
ALS	**2400508 Сегодня. [Инфо]***	240081 Факты. [Инфо]*	2400480 ТСН. [Инфо]*	2400467 'Викна-Новости'. [Инфо]*	20088 'Завтрак с 1+1'. Информацион... [Развлечения]*
CBF	200477 'Завтрак. Выходной'. [Развлечения]	500315 Ток-шоу 'Касается каждого'. [Развлечения]*	400357 'Холостяк', 10 сезон, 7 эп.,... [Развлечения]*	400373 'Холостяк'. Реалити-шоу, 7 с... [Развлечения]	6000473 Половинки, 3 сезон, 3 эп. [Развлечения]
repeat_cov	**2400508 Сегодня. [Инфо]***	**5900256 Утро с Украиной. [Инфо]***	240081 Факты. [Инфо]*	2400480 ТСН. [Инфо]*	2400467 'Викна-Новости'. [Инфо]*
final (hybrid_v4_cov80)	**2400508 Сегодня. [Инфо]***	240081 Факты. [Инфо]*	**5900256 Утро с Украиной. [Инфо]***	2400467 'Викна-Новости'. [Инфо]*	2400480 ТСН. [Инфо]*
Light viewer (thin history): user 3799694907521863650 — History weeks 11-20: 2.6 viewing hours, 3 distinct labelled shows watched. Target weeks 21-30: 5 shows in the cov50 top-5, 5 in the cov80 top-5.					
actual cov50 top-5	**700389 Кто сверху? [Развлечения]**	**2400480 ТСН. [Инфо]***	**12002733 М/с 'Буба'. [Детям]**	**400356 'Я стесняюсь своего тела. Ук... [Познавательное]**	**90044740 Х/ф 'Бриллиантовая рука'. [Фильмы]**
popularity	**2400480 ТСН. [Инфо]***	240081 Факты. [Инфо]	2400508 Сегодня. [Инфо]	20088 'Завтрак с 1+1'. Информацион... [Развлечения]	6500479 'Мир наизнанку'. [Познавательное]*
ALS	400361 'МастерШеф'. [Познавательное]	400335 'Хата на тата'. [Развлечения]	400357 'Холостяк', 10 сезон, 7 эп.,... [Развлечения]	2400467 'Викна-Новости'. [Инфо]	6500479 'Мир наизнанку'. [Познавательное]*
CBF	400455 Любовь на выживание, 3 сезон... [Познавательное]	6500479 'Мир наизнанку'. [Познавательное]*	400426 'Меняю жену'. [Развлечения]*	6500485 Премьера. 'Невероятно интере... [Познавательное]	**2400480 ТСН. [Инфо]***
repeat_cov	**2400480 ТСН. [Инфо]***†	240081 Факты. [Инфо]†	2400508 Сегодня. [Инфо]†	20088 'Завтрак с 1+1'. Информацион... [Развлечения]†	6500479 'Мир наизнанку'. [Познавательное]*†
final (hybrid_v4_cov80)	**2400480 ТСН. [Инфо]***	2400467 'Викна-Новости'. [Инфо]	12001682 М/с 'Губка Боб Квадратные Шт... [Детям]	240081 Факты. [Инфо]	20088 'Завтрак с 1+1'. Информацион... [Развлечения]

Familiar vs new. For the heavy viewer every method except CBF recommends only programmes the user already watched; the user's true top-5, however, contains two children's series (*Удивительный мир Гамбола*, *Время приключений*) that no method proposed, possibly watched by another household member on the same account. For the light viewer repeat_cov has nothing of its own and falls back to popularity in all five slots, while the hybrid fills them from its own candidates and finds the one hit, *ТСН*, like popularity does.

How the methods differ. ALS and popularity both put the big news shows first: on this data a low-rank model mostly learns popularity (its best configuration has only 8 factors). CBF looks for similar metadata and proposes new entertainment shows, none of which the users watched. Repeat models know only the user's own history; slot habits know only the user's channels and hours. The re-ranker uses both: in the hybrid_v2 setup the slot rank carries 44.7% of the total gain and the repeat rank 15.1% (`docs/hybrid_cov50_feature_importance.md`). Association rules and personalized PageRank were not implemented; conceptually, rules would add "after A, B" co-viewing and PageRank would spread a history to neighbouring programmes, i.e. towards discovery, the direction in which ALS and CBF already did worse here.

Decision. For sweet.tv I would choose the two-layer strategy: habit-based candidates (repeat counts on coverage, slot habits, popularity) re-ranked by a learned model. It is the best on validation and on the leaderboard (Table II), and it degrades gracefully for thin histories. Its cost is the honest trade-off: repeat_cov runs in about 1 s and reaches 0.4080 on validation, the final ensemble needs about 21 minutes for 0.4265; a service could serve repeat_cov as the always-on default and refresh the re-ranker offline. For the user experience the lists are mostly familiar programmes; this matches how linear TV is watched, but it is not discovery, and nothing in the data tells whether a recommended new programme would have been enjoyed. Limitations: only 10.8% of airtime has an id, so the remaining viewing is invisible as items; the hidden label is still only approximated (§5); absence from the log does not mean a user never saw a programme elsewhere.

5. Discussion

[PAVEL: draft below, to be rewritten in my own words]

Failure case: a validation label that fitted the leaderboard and was wrong. I first counted a programme as watched on *any* overlap and ranked by distinct days (`n_days`). Three submissions (popularity, two repeat models) fitted this label almost perfectly: validation gaps 1.297 / 1.425 / 1.098 against leaderboard gaps 1.302 / 1.441 / 1.107, coefficient of variation of the LB/val ratio 0.0048 (`docs/target_definition.md`). I selected models on it for three stages.

Root cause. All three submissions ranked programmes by any-overlap counts, so they were bound to agree with an any-overlap label; the comparison could rank candidate labels only among similar models and could not test a definition none of them used. A diagnostic submission built on a different definition (repeat_cont50, > 50% continuous coverage) settled it: the any-overlap label predicted 0.298, the coverage labels 0.380-0.387, and it scored 0.417 public / 0.400 private. With four points, `n_days` became the worst candidate (cv 0.156) and cov50 the best (cv 0.057), though no candidate fits well: even for cov50 the ratio rises from 0.93 (popularity) to 1.09 (repeat_cont50).

Fix and verification. I switched selection to cov50, re-tuned repeat on it (0.3841 → 0.4080) and retrained the hybrid on coverage labels; the public score went from 0.36752 (repeat_v2) to 0.41706 (repeat_cont50) and 0.45010 (first hybrid on coverage labels). A second, smaller error: converting epoch seconds back to timestamps went through the DuckDB session time zone (Europe/Moscow), so all day counts started at 21:00; 2.1% of (user, programme, week) day counts changed after the fix (`docs/errata.md`). A third finding is a limitation: the LightGBM hybrids are not bit-for-bit reproducible (validation 0.4271 / 0.4260 / 0.4263 for the hybrid_v2 setup in three runs; a regenerated `final.csv` matches the scored one in 824 of 1,260 rows, 96.9% of ids). The likely cause, not verified, is tie-breaking in rank features on rows whose order DuckDB does not fix.

What worked.

- A cheap diagnostic submission whose outcome I wrote down in advance; it was worth more than the whole label study.
- Checking the join against a brute-force version before building anything on it.
- The temporal check: in the first hybrid the binary model looked best but lost most when trained only on earlier weeks (0.4660 → 0.4521 vs 0.4656 → 0.4581 for LambdaRank, any-overlap label, `docs/hybrid_results.md`), so I used LambdaRank.

What surprised me or did not work.

- Both required methods stayed below popularity (ALS 0.2528, CBF 0.1154 vs 0.2752): on linear TV, habit beats similarity.
- Slot habits are strong alone (0.4015) but added little on top of repeat in the hybrid.
- Recency (five weighted folds) was invisible on validation (0.4270 vs 0.4272) but every later refit, which also had more candidates and features, scored 0.007-0.012 higher on the private leaderboard than hybrid_v2 (0.43259); the data cannot separate the share of recency.

Next improvement. Make the rank features deterministic (sort before ranking) and re-run the final setup twice; only then compare variants that differ by less than 0.002.

6. AI Usage Disclosure

AI tools used. Claude Code (Anthropic), versions 2.1.296, 2.1.308, model claude-opus-5-5 (Claude Opus 5.5): one session from 22 Oct 2026 16:01 to 24 Oct 2026 13:16 MSK (with a break between 22 and 24 Oct), 16 prompts from me plus 1 message sent while the agent was working, and 273 tool calls (181 Bash, 35 Write, 25 Read, 12 Edit, 12 WebFetch, 4 Monitor, 3 ToolSearch, 1 Skill), counted from the transcript when this PDF was built; the transcript continues with the final commit and push. Claude (Anthropic, claude.ai chat), outside Claude Code: planning the stages, drafting my prompts, finding and reading the prize-winning write-up together with me, and the report plan (`docs/report_plan.md`).

How AI was used. Code generation for every module in `src/`, data inspection and debugging (the time-zone bug, the user-id overflow), experiment runs and logging, reference checks (DOIs and URLs), figures and a first draft of this report (Claude Code); planning and prompt drafting (Claude).

What I personally verified. **[PAVEL: e.g. the hand-computed AP@5 in reports/hand_check.md; the Kaggle scores against the Submissions page; that I opened and read the write-up [2] (22 Oct. 2026); ...]**

What I trusted without verification. The aggregate numbers in Tables I-III and the logs were computed by agent-written code that I did not re-implement; the agent's join check and unit tests are the only independent checks of that code. The metadata of references [3]-[6] was checked by the agent through DOI/Crossref and publisher pages, not read in full by me. The agent could not open [2] (HTTP 403); I verified it myself. The cause given for the reproducibility limit is a hypothesis.

Session log reference. `manuilov_exam_session.jsonl` is the unmodified Claude Code transcript of this session.

References

[1] Kaggle, "sweet.tv - TV program recommender," community prediction competition, 2020. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender. [Accessed: Oct. 24, 2026].

[2] N. Melentev, "Two-layered recommender system methodology: a prize-winning solution," *Towards Data Science*, Mar. 20, 2021. [Online]. Available: https://towardsdatascience.com/two-layered-recommender-system-methodology-a-prize-winning-solution-c4bdb815b156. [Accessed: Oct. 22, 2026].

[3] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 263-272. [Online]. Available: https://doi.org/10.1109/ICDM.2008.22. [Accessed: Oct. 24, 2026].

[4] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. [Online]. Available: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html. [Accessed: Oct. 24, 2026].

[5] C. J. C. Burges, "From RankNet to LambdaRank to LambdaMART: An overview," Microsoft Research, Tech. Rep. MSR-TR-2010-82, 2010. [Online]. Available: https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/. [Accessed: Oct.

24, 2026].

[6] P. Cremonesi, Y. Koren, and R. Turrin, "Performance of recommender algorithms on top-N recommendation tasks," in *Proc. 4th ACM Conf. Recommender Systems (RecSys)*, 2010, pp. 39-46. [Online]. Available: https://doi.org/10.1145/1864708.1864721. [Accessed: Oct. 24, 2026].

[7] "Implicit: Fast Python collaborative filtering for implicit datasets," documentation (v0.7.2 online; v0.7.3 used). [Online]. Available: https://benfred.github.io/implicit/. [Accessed: Oct. 24, 2026].

[8] LightGBM, "Parameters," LightGBM documentation. [Online]. Available: https://lightgbm.readthedocs.io/en/latest/Parameters.html. [Accessed: Oct. 24, 2026].

[9] DuckDB, "Documentation." [Online]. Available: https://duckdb.org/docs/. [Accessed: Oct. 24, 2026].